
 PLAYBOOK FOR INTERN CANDIDATES

Spec-Driven

Development

A practical reference on how Nexlab approaches software development in the era of capable coding agents. Written for intern candidates to study during the 3-day entrance test — and to argue with.

Reference, not mandate.

Tài liệu này tổng hợp cách Nexlab đang vận hành Spec-Driven Development — phương pháp build phần mềm với AI là partner, không chỉ là công cụ assist. Mục đích là cho intern candidate có một starting point để hiểu vấn đề mà ngành software đang đối mặt trong kỷ nguyên AI, và cách Nexlab đã chọn để xử lý nó.

Đối tượng đọc

Tài liệu này được viết cho intern candidate đang làm bài test 3 ngày của chương trình Nexlab AI Software Engineer. Bạn không phải đọc 100% trước khi start. Bạn có thể skim Section 01–02 để nắm bối cảnh, sau đó dive sâu vào Section 03–05 khi thực hành PH-0 và PH-1 trong Day 2 của bài test.

Tinh thần tiếp cận

Đây là tài liệu **tham khảo**, không phải doctrine. Nexlab tin rằng phương pháp tốt được hình thành qua tranh luận, không qua tuân thủ. Khi bạn đọc và thấy điều gì không hợp lý — chính là lúc nên viết xuống. Bài test khuyến khích bạn critique, đề xuất alternative, và defend lập trường riêng.

SDD là một methodology, không phải một sản phẩm cụ thể. Trong tài liệu này, mọi đoạn nhắc đến công cụ (Claude Code, Codex, Cursor, Aider, v.v.) đều có thể thay thế bằng coding agent khác bạn thấy fit. Tên cụ thể chỉ là ví dụ.

Cách document được tổ chức

Tám sections, mỗi section ~1 trang. Section 01–02 nói về *tại sao*. Section 03–05 nói về *cách làm* giai đoạn early-phase. Section 06–08 cung cấp thêm context về tool ecosystem, common pitfalls, và glossary.

VOICE NOTE

Nexlab viết theo bốn nguyên tắc voice: Substantive (có data, không slogan), Precise (dùng đúng thuật ngữ), Honest (không phóng đại), Patient (viết cho người đọc 5 năm sau, không phải reader đang scroll trên LinkedIn).

What's inside.

Tám sections, mỗi phần khoảng một trang đọc. Trang được đánh số ở footer dưới cùng.

01	Tại sao SDD tồn tại	04
	Vấn đề của ngành software trong kỷ nguyên AI · Phân biệt với TDD, BDD · Nguyên tắc gốc	
02	Bảy phase từ ý tưởng đến production	05
	PH-0 đến PH-6 · Đầu là spec work, đầu là implementation · Gate giữa các phase	
03	PH-0 — Problem Discovery	06
	Phỏng vấn stakeholder · Validation criteria · Khi nào pivot · Mom Test reference	
04	PH-1 — Product Spec	07
	Cấu trúc PRD · 8 sections bắt buộc · Must / Should / Could · Out of Scope	
05	Acceptance Criteria — quy tắc và ví dụ	08
	Format WHO → ACTION → RESULT · Ví dụ đúng/sai · Cách viết AC testable	
06	Coding agents — bạn chọn tool nào	09
	Claude Code · Codex · Cursor · Aider · Config file tương ứng · SDD là methodology, không lock-in	
07	Bảy lỗi phổ biến intern mắc	10
	Từ skip phỏng vấn đến vibe-prompting · Cách tránh · Tín hiệu nhận biết	
08	Glossary và đọc thêm	11
	Thuật ngữ Nexlab dùng · References từ Mom Test đến Diátaxis · Self-study tiếp sau bài test	

Tại sao Spec-Driven Development tồn tại.

Trước AI: vấn đề là code chậm. Sau AI: vấn đề là code nhanh hơn khả năng nghĩ. Phương pháp xây phần mềm phải thay đổi theo.

Vấn đề trong kỷ nguyên AI

Trước khi LLM-based coding agents trở nên capable, bottleneck của một dự án software là **tốc độ viết code**. Junior dev có thể tốn 2-3 ngày cho một feature mà senior viết trong 2 giờ. Toolchain, framework, library lành mạnh đều xoay quanh việc giúp developer code nhanh hơn và ít lỗi hơn.

Năm 2025-2026 thì bottleneck đã dịch chuyển. Một agent như Claude Code hoặc Codex có thể implement một feature trung bình trong vài phút — nhanh hơn cả senior. Nhưng chất lượng output phụ thuộc gần như hoàn toàn vào **chất lượng spec** được cung cấp. Spec mơ hồ → agent hallucinate business rules → bug đến production. Spec đầy đủ → output usable trong commit đầu.

Spec-Driven Development sinh ra từ quan sát đó. Thay vì tối ưu cho tốc độ code, SDD tối ưu cho **chất lượng spec trước khi code**. Phương pháp này không mới — Joel Spolsky đã viết về Painless Functional Specifications từ năm 2000. Cái mới là: trong môi trường có coding agent capable, giá trị của spec đã tăng lên 10 lần.

Phân biệt với các methodology lân cận

Methodology	Tinh thần	Khác với SDD ra sao
TDD — Test-Driven Development	Viết test trước, code sau, refactor	TDD focus vào test cấp unit. SDD focus vào spec cấp business + acceptance.
BDD — Behavior-Driven Development	Mô tả behavior bằng Given-When-Then	BDD là một subset format của SDD. SDD bao trùm cả PRD, ADR, system prompts.
Spec-by-Example	Spec qua ví dụ cụ thể, không generalize	Spec-by-Example là một kỹ thuật trong SDD. SDD bao gồm phương pháp tổ chức tài liệu lớn hơn.
Waterfall	Spec đầy đủ trước khi code, không thay đổi	Waterfall giả định spec không thay đổi. SDD chấp nhận spec sẽ thay đổi nhưng có process chính thức để track.

NGUYÊN TẮC GỐC

Spec trước · Code sau · Gate không thỏa hiệp.

Ba mệnh đề ngăn chừng đủ. Spec phải có trước khi viết dòng code đầu. Code phải tuân theo spec, không phải ngược lại. Mỗi phase có gate, gate không pass thì không sang phase kế.

Từ ý tưởng đến production, qua bảy phase.

Mọi dự án Nexlab đi qua bảy phase, thứ tự cố định, timeline co giãn. Bốn phase đầu là spec work — không có dòng code nào. Code chỉ bắt đầu ở phase thứ năm.

PH-0	PH-1	PH-2	PH-3	PH-4	PH-5	PH-6
Discovery	PRD	Tech Arch	Design	SDD Setup	Sprint	Deploy

Phase với gạch ngang trên màu cam là spec work. Gạch đen là implementation và deploy.

Phase	Tên	Output chính	Gate pass khi	Thời gian
PH-0	Problem Discovery	docs/PROBLEM.md	≥ 6/10 users xác nhận problem tồn tại	1-3 ngày
PH-1	Product Spec (PRD)	docs/PRD_v1.md	AC testable, Out of Scope rõ	2-5 ngày
PH-2	Technical Architecture	docs/TechArch_v1.md + ADRs	Stack rõ, designer unblock được	2-3 ngày
PH-3	Design & UI System	design-system/complete	≥ 80% screens có pages spec	3-7 ngày
PH-4	SDD Setup	8 files committed	npm test FAIL có ý nghĩa	1-2 ngày
PH-5	Sprint Implementation	Working code + tests passing	npm test ALL PASS	1-4 tuần
PH-6	Deploy & Go-live	Production URL + monitoring	Manual smoke + UAT pass	1-2 ngày

Tại sao thứ tự cứng?

PH-2 trước PH-3. Designer cần biết frontend framework để dùng đúng component library và data model. Quyết định stack sau khi đã design → design phải làm lại.

PH-4 trước PH-5. Coding agent đọc CLAUDE.md (hoặc AGENTS.md, .cursorrules tùy tool) ở mỗi session. Thiếu file này → agent tự quyết kiến trúc inconsistent. Test seeds phải FAIL có ý nghĩa trước khi implement — vì expected values lấy từ spec, không từ code.

ĐỪNG ĐẢO THỨ TỰ

"Code trước để thấy hướng đi, viết spec sau" là nguyên nhân phổ biến nhất khiến MVP thất bại — không phải vì ý tưởng sai, mà vì implementation không nhất quán và không validate được core assumption.

PH-0

Problem Discovery.

Một đến ba ngày tại đây tiết kiệm ba đến sáu tuần build sai sau này. Mục tiêu duy nhất của PH-0 là *xác nhận problem thực sự tồn tại* trước khi đầu tư bất cứ thứ gì khác.

Output cuối cùng

Một file duy nhất: `docs/PROBLEM.md`. Sáu sections, mỗi section trả lời một câu hỏi cụ thể về problem space.

VẤN ĐỀ Hai đến ba câu mô tả vấn đề cốt lõi. Có số liệu càng tốt.

STAKEHOLDERS Ba đến năm người/role bị ảnh hưởng. Mỗi người có pain riêng.

ĐAU RA SAO Cụ thể với số. Không "khó khăn" chung chung.

TẦN SUẤT Daily / weekly / monthly với volume.

SOLUTION HIỆN TẠI Người ta đang xử lý ra sao. Đây là "đối thủ" thực sự.

GAP Solution hiện tại còn thiếu gì.

Validation criteria

Một PROBLEM.md tự nó không đủ — phải có evidence từ user thật. Nexlab dùng bốn check trước khi gate sang PH-1:

Item	Cách validate	Pass khi
Problem tồn tại	Phỏng vấn 7-10 target users thật	≥ 6/10 xác nhận đang gặp vấn đề này
Willing to act	Hỏi về current workaround: tốn bao nhiêu tiền/giờ	User đang thực sự tốn resource cho workaround
MVP scoped đúng	Đọc MVP boundary đề xuất	Build được trong 4-6 tuần với 1 dev
Market timing	Check regulatory / industry trigger	Có urgency rõ, không "nice to have"

Câu hỏi phỏng vấn — behavioral, không hypothetical

Một quy tắc duy nhất từ Mom Test: hỏi về **hành vi đã xảy ra**, không hỏi về **ý kiến hoặc dự định**. Lý do: user lịch sự sẽ nói "yes" cho mọi câu hỏi giả định, dữ liệu đó không actionable.

SAI — HYPOTHETICAL

"Bạn có muốn có tool tự động hóa việc X không?"

"Nếu có app làm việc này, bạn có dùng không?"

ĐÚNG — BEHAVIORAL

"Tuần trước bạn dành bao lâu để làm việc X?"

"Lần gần nhất gặp tình huống Y, bạn xử lý ra sao?"



PH-1

Product Spec (PRD).

PRD trong SDD không phải tài liệu 50 trang. Là tài liệu đủ để coding agent implement đúng — không thừa, không thiếu. Focus vào Must stories và Acceptance Criteria testable.

Tám sections bắt buộc

Đây là cấu trúc Nexlab dùng. Bạn có thể adapt, nhưng tám phần này có lý do tồn tại — bỏ phần nào sẽ tạo gap downstream.

1 · PRODUCT OVERVIEW	Ba đến năm câu. Vấn đề gì, ai dùng, giải quyết thế nào. Không slogan.
2 · PERSONAS	Một đến hai personas với context cụ thể: role, tech literacy, device, frequency of use.
3 · SUCCESS METRICS	KPI đo được, có baseline so sánh. Không "tăng engagement".
4 · USER STORIES	Nhóm theo epic. Priority Must / Should / Could. Format: Là [ai], tôi muốn [hành động], để [benefit đo được].
5 · ACCEPTANCE CRITERIA	Mỗi Must story phải có AC. Format bắt buộc: WHO → ACTION → RESULT. Chi tiết Section 05.
6 · OUT OF SCOPE	Liệt kê + lý do defer + milestone sẽ làm. Đây là phần quan trọng — định nghĩa cái <i>không</i> làm rõ hơn cái làm.
7 · ASSUMPTIONS	Giả định đang dựa vào để build. Mỗi cái có thể bị disprove sau.
8 · OPEN QUESTIONS	Câu hỏi chưa quyết định. Phải resolve trước PH-4 SDD Setup.

Quy tắc về priority

Must / Should / Could là một subset của MoSCoW (Must / Should / Could / Won't). Nexlab bỏ Won't vì đã có Section 6 Out of Scope đảm nhận.

- **Must** — Không có → product không launch được. Cover trong sprint đầu.
- **Should** — Quan trọng nhưng có workaround tạm thời. Thường vào sprint 2-3.
- **Could** — Nice to have. Thường defer ra phase sau.

Một PRD MVP healthy thường có 5-8 Must stories, 3-5 Should, và Could đẩy hết sang OOS.

CẢNH BÁO PHỔ BIẾN

Nếu PRD có hơn 12 Must stories — gần như chắc chắn scope quá lớn cho MVP. Quay lại và downgrade một số sang Should hoặc OOS. Better launch một MVP nhỏ và iterate, hơn launch MVP cồng kềnh và miss timeline.

Một format. Không ngoại lệ.

Acceptance Criteria là sợi dây kết nối spec với test seed, và test seed với code. AC viết tốt thì test seed viết được; test seed viết được thì coding agent implement đúng. AC viết kém ở đầu sẽ sinh ra bug ở cuối.

Format bắt buộc

FORMAT

WHO trigger · **ACTION** cụ thể · **RESULT** quan sát được (số / text / state)

Ba mệnh đề. Mỗi mệnh đề trả lời một câu hỏi:

- **WHO** — Ai/cái gì kích hoạt? User chưa đăng nhập? Admin? Hệ thống cron job? Phải specific.
- **ACTION** — Hành động cụ thể, không trừu tượng. "Submit form" cụ thể hơn "tương tác với trang".
- **RESULT** — Có thể đo bằng số, đọc bằng text, hoặc quan sát bằng state. Phải `expect(...).toBe(...)` được.

Ví dụ — đúng và sai

SAI — KHÔNG TESTABLE

"Form validation hoạt động đúng."

Thiếu cả ba phần. Không có WHO, không có ACTION cụ thể, không có RESULT đo được.

ĐÚNG — TESTABLE

"Khi user submit email không có ký tự @ → hiển thị inline error 'Email không hợp lệ' bên dưới input, border ô email đổi sang đỏ, trong 200ms."

SAI — VAGUE

"Hệ thống phải xử lý nhanh."

"Nhanh" là gì? 100ms? 1 giây? Cho ai? Với volume bao nhiêu? Không thể test.

ĐÚNG — CÓ SỐ

"Khi user search với từ khóa ≤ 50 ký tự → kết quả trả về trong p95 ≤ 800ms với dataset 10.000 records, sort mặc định theo relevance."

SAI — THIẾU WHO

"Order phải được lưu vào database và gửi email confirmation."

Ai trigger? User? Admin? Auto-process?

ĐÚNG — ĐẦY ĐỦ

"Khi customer click 'Place Order' và payment success → order record tạo với status='pending', email gửi tới customer.email trong 60s với subject 'Order Confirmed #[order_id]'."

Từ AC sang test seed

AC viết đúng thì test seed gần như tự sinh ra. Ví dụ AC bên trái → test seed bên phải:

```
// AC: Khi user submit email không có @ → hiển thị error
// 'Email không hợp lệ' trong 200ms, border ô email màu đỏ
```

```
describe('email validation', () => {
  it('rejects email without @ within 200ms', async () => {
    const start = Date.now()
```


SDD là methodology. Tool là lựa chọn của bạn.

Tài liệu này nhắc đến CLAUDE.md nhiều lần vì Nexlab có lịch sử dùng Claude Code lâu nhất. Nhưng SDD không phụ thuộc vào agent cụ thể. Bạn dùng Codex, Cursor, Aider, hoặc tool nào khác — phương pháp vẫn áp dụng.

Bốn coding agents phổ biến

Agent	Vendor	Config file	Đặc điểm so với agent khác
Claude Code	Anthropic	CLAUDE.md + .claude/settings.json	Mạnh ở long-horizon task, tool use chuẩn MCP, hooks system
Codex	OpenAI	AGENTS.md	Mạnh ở interactive REPL, fast iteration, GPT model family
Cursor	Cursor (Anysphere)	.cursorrules + .cursor/rules/*.mdc	IDE-integrated, multi-model, mạnh ở context-aware suggestion
Aider	Open source	CONVENTIONS.md + .aider.conf.yml	CLI-first, git-native commits, model-agnostic

Chuẩn AGENTS.md đang nổi lên

AGENTS.md là một *open standard* đang được nhiều agent adopt như format chung cho project config — tương tự cách README.md trở thành chuẩn de facto cho mọi repo. Một số agent đọc cả AGENTS.md lẫn config riêng của họ. Bạn có thể xem thêm tại [agents.md](#).

Trong dài hạn, có thể CLAUDE.md / .cursorrules / CONVENTIONS.md đều converge về AGENTS.md. Hiện tại 2025-2026 thì mỗi agent vẫn có format riêng — nên project Nexlab thường có 1-2 file primary cộng AGENTS.md mirror cho compatibility.

Triết lý chọn agent

Chọn agent dựa vào ba câu hỏi:

- Workflow của team là gì?** IDE-heavy (Cursor) vs terminal-heavy (Aider, Claude Code CLI) vs chat-heavy (Codex / Claude.ai)
- Long-horizon hay short iteration?** Build feature lớn one-shot (Claude Code) vs nhiều turn nhỏ (Codex, Aider)
- Cost và privacy constraint?** Mỗi vendor có pricing model và data policy khác. Project enterprise có thể cần on-prem (Aider với local model)

LƯU Ý CHO INTERN

Trong bài test 3 ngày, bạn không bị bắt dùng Claude Code. Pick agent quen tay nhất, demo hiệu quả nhất. Reviewer quan tâm cách bạn áp dụng SDD methodology, không quan tâm bạn dùng vendor nào.

Bảy lỗi intern thường mắc, và cách tránh.

Các lỗi dưới đây Nexlab đã quan sát qua nhiều batch onboarding. Không phải vì intern thiếu khả năng — mà vì pattern mặc định từ trường ĐH và công việc dev truyền thống chưa fit với môi trường AI-first.

1 • Skip phỏng vấn, làm spec dựa vào "common sense"

Triệu chứng: PROBLEM.md viết được trong 30 phút, dựa hoàn toàn vào assumption. Không có quote từ user thật. Hậu quả: spec dựa vào model nội tâm của bạn, không phải vấn đề thực tế. Tránh bằng cách: dù mock project, vẫn phỏng vấn 1-2 người thật. Ba mươi phút phỏng vấn tốt hơn ba giờ brainstorm một mình.

2 • Vibe-prompting thay vì spec-prompting

Triệu chứng: prompt vào agent kiểu "build me an app for X". Agent improvise toàn bộ — đôi khi hay, đôi khi lệch hoàn toàn yêu cầu. Tránh bằng cách: cung cấp spec đầy đủ (PRD section liên quan, AC, types) cho agent ngay từ đầu. Agent là partner cần thông tin, không phải oracle đọc được ý bạn.

3 • AC mơ hồ, không testable

Triệu chứng: AC kiểu "hệ thống phải nhanh", "form validation hoạt động đúng". Test seed không viết được. Coding agent tự diễn dịch — và diễn dịch sai. Tránh bằng cách: với mỗi AC, hỏi "tôi có thể viết `expect(result).toBe(expected)` không?" Nếu không — viết lại.

4 • CLAUDE.md (hoặc config file tương đương) quá dài

Triệu chứng: file 300-500 dòng, chứa cả history project, cả thông tin đã outdated. Agent đọc mỗi session — chiếm context window, ít chỗ cho task. Tránh bằng cách: target ≤ 150 dòng. Với mỗi dòng, hỏi: "nếu agent không biết điều này, có quyết định sai không?" Có → giữ. Không → cắt.

5 • Tự sửa expected values trong test để pass

Triệu chứng: test FAIL → thay vì sửa code, đổi expected value cho match. Hoặc tệ hơn: bảo agent "test này đang FAIL, fix it" mà không specify fix code hay fix test. Tránh bằng cách: test expected values lấy từ spec, không lấy từ code. Nếu spec sai → fix spec rồi sửa test theo. Không bao giờ ngược lại.

6 • Bỏ qua Out of Scope

Triệu chứng: PRD không có Section 6 Out of Scope, hoặc OOS chỉ liệt kê "phase sau làm". Hậu quả: scope creep diễn ra mọi sprint vì không có baseline để pushback. Tránh bằng cách: mỗi item OOS phải có (a) lý do defer (b) milestone sẽ làm hoặc condition để revisit.

7 • Code trước, spec sau, "để thấy hướng đi"

Triệu chứng: cảm thấy "phải code mới thấy được nó hoạt động ra sao". Bắt đầu prototype, dần dần code dày lên, spec không bao giờ được viết. Tránh bằng cách: nhận ra rằng "thấy hướng đi" là phase PH-0 và PH-1, không phải PH-5. Nếu cần prototype để clarify — prototype giấy hoặc Figma, không prototype code.

Từ điển và đọc thêm.

Thuật ngữ Nexlab dùng và các nguồn để self-study sau bài test. Tất cả đều khả thi đọc trong 1-2 buổi.

Glossary

SDD	Spec-Driven Development. Phương pháp viết spec đầy đủ trước khi viết code, dùng spec để hướng dẫn coding agent.
AC	Acceptance Criteria. Tiêu chí cụ thể để verify một user story đã được implement đúng. Format Nexlab: WHO → ACTION → RESULT.
ADR	Architecture Decision Record. Tài liệu một trang ghi lại một quyết định kiến trúc lớn: context, decision, consequences, alternatives.
CLAUDE.md / AGENTS.md	Project constitution file. Coding agent đọc ở mỗi session để biết stack, schema, business logic, conventions. Target ≤ 150 dòng.
PRD	Product Requirements Document. Tài liệu spec sản phẩm. Nexlab dùng cấu trúc 8 sections.
TEST SEED	Test case viết từ AC, expected values lấy từ spec. Phải FAIL trước khi implement.
GATE	Tiêu chí cụ thể để pass từ phase này sang phase tiếp. Không pass → không sang.
MCP	Model Context Protocol. Chuẩn open của Anthropic để coding agent giao tiếp với tool và data source bên ngoài.

Đọc thêm — ưu tiên theo thời gian đầu tư

30 PHÚT — ESSENTIALS

- **The Mom Test** by Rob Fitzpatrick — đọc chapter 2 và 3 để hiểu cách phỏng vấn user behavioral.
- **Joel Spolsky — Painless Functional Specifications** (4 part series, 2000) — gốc của spec-first thinking.

2-3 GIỜ — DEEPER

- **Diátaxis framework** (diataxis.fr) — bốn loại documentation và khi nào dùng cái nào.
- **Michael Nygard — Documenting Architecture Decisions** (2011) — ADR format gốc.
- **Anthropic — Building Effective Agents** (2024) — patterns cho agent design.

1 NGÀY — INDUSTRY CONTEXT

- **IIBA BABOK** — section "Elicitation and Collaboration" — formal framework cho stakeholder elicitation.
- **GitHub Spec-Kit documentation** — một framework SDD opinionated khác để so sánh với cách Nexlab làm.
- **Aider conventions documentation** — best practices cho CONVENTIONS.md, nguyên tắc tương tự CLAUDE.md.

— END OF PLAYBOOK

Spec trước. Code sau. Gate không thỏa hiệp.

Đây là ba câu Nexlab nhắc lại nhiều nhất, không vì chúng catchy, mà vì chúng tóm tắt cả phương pháp. Bạn sẽ thấy chúng lặp lại trong mọi tài liệu, mọi sprint review, mọi cuộc trò chuyện về chất lượng.

Bài test 3 ngày của bạn là cơ hội đầu tiên để áp dụng — và quan trọng hơn, là cơ hội để pushback nếu bạn thấy có gì không hợp lý.